Deliverable Five

Final Report

Group 19

LEXXE

José Ribeiro (40745880)

Matthew Seery (44760442)

Oscar Gardner (44917651)

Thomas Pearce (43270484)

Tim Chard (44535686)

Table of Contents

Introduction

Pace has been an amazing experience for our group that has not only given us the opportunity to experience to work on technical projects within an established company (LEXXE) but has also provided the opportunity to better understand the theory learnt in various prior units, the processes involved in delivering software, the importance of the client relationship and the benefits and challenges of working in teams.

The overwhelming consensus in Group 19 is that the Pace unit has been an immensely valuable experience. The value of collaboratively working with both fellow students and client toward achieving a common objective has provided an opportunity to better understand what will soon become a regular occurrence as we join the workforce and become productive members of society.

The various deliverables required throughout the project provided the means to ensure the final deliverable could be successfully complete. Each of the deliverable had a direct relationship with the software development life cycle, it included planning the project, determining and analysing requirements, design, testing, implementation and consideration for future maintenance.

Project planning ensured the team would not only put names and dates against activities, but ensured coverage for the entire scope of the project, it helped highlight missing activities and over allocation of resources that if left unchecked would have impacted on the final deliverable.

The requirements analysis phase ensured that all members of the team will clear on the clients requirements ensuring these were appropriately tracked and assessed for impact on design, implementation and testing.

The design phase guided the team to determine the most appropriate way to meet the clients requirements and ensured the team had clear actionable items to be handled as part of implementation for which progress could be monitored.

The implementation phase focused the team on the building the technology based on the agreed upon design that through the continuous improvements on prototypes, testing and client feedback would ultimately become the final deliverable - the product.

Much has been learnt throughout the various stages of the project. The following sections of this document aims to reflect on the Group 19’s experience with each of the phases, the units learning outcomes and conclude with the groups experience with the PACE unit as a whole.

Project Planning

Much planning was done during the initial stage of the project, but planning was not a one off activity. In fact, planning was an integral part of all deliverables that involved all group members. It was not only an activity undertaken at the start of each deliverable but one that was ongoing to allow tracking of work items and timelines to ensure adjustment were made when the deliverables were not tracking to plan.

For each of the deliverables, the entire group met together to determine what had to be done to complete the upcoming deliverable, who would be responsible for each of the work items identified, interdependencies amongst work items and when each item would need to be delivered to ensure each phase was successfully complete by the due date. Additionally, the team was in constant communication to allow tracking of progress but also resolve any matters that had not been taken into consideration previously.

Because the project was split into various phases and the deliverables of each phase was clearly documented, identifying the activities that needed to take place was a reasonably straight forward activity. For development purposes, the team chose an agile approach that allowed the team to focus on specific features and prioritize work items as required, including when the client decided to changed requirements, something we experienced on a couple of occasions.

In our group, work item assignment was made easy due to the composition of the team. We had members that leaned more to the technical side (the developers) and those that were less technically inclined that focused on activities such as documentation, testing and planning. There was never a time where concerns were raised around who would do what. To us, this clearly highlighted the importance of having the right balance of skills within a team.

Although team members had their own set of activities to complete, it soon became evident that team members could not simply complete their work in isolation. For example, documentation required for each phase could not be completed without ensuring that the various documents or sections within documents integrated successfully as a whole. This was not the case initially and it soon became apparent that ongoing communication was necessary. The group decided to use Google Docs as a means to collaboratively work on documentation, this helped ensure harmony across the various sections. The team also decided to use Slack for real time online communication enabling anytime, anywhere communication that sped up clarifications and decision making.

The team often committed to complete work items on a specific schedule that was based on initial estimates. Although effort was put into the estimation process, it was not always precise and work items were not always completed by their due date. At times this was cause for frustration, in particular for those that depended on the completion of delayed

items to complete their own work. To ensure the likelihood of this being a recurring theme was greatly reduce, our group took steps toward reducing this risk. The first of these was to ensure that it assessed the previous deliverable to determine what was done well and what could have done better - this allowed the group as whole to discuss delays and come up with ways to prevent slipping on dates, one such item was to schedule deliverable with a buffer that allowed for slippage. The second was to regularly check on ongoing activities which highlighted potential delays early and allowed the group to determine corrective action.

At the completion of the PACE unit, it has become clear to the group that planning is an important part of the project. Planning provided the processes by which work items were identified, assigned, estimated, scheduled and tracked. Planning increases the likelihood of successfully delivering the product on time.

Requirements and Analysis

The purpose of this project was to enhance the current capabilities of Lexxe's News and Mood app. The current system provides push notifications for breaking and user selected keywords. However, the biggest problems that needed to be addressed were the sending of redundant information, the timeliness of the push notifications, the accuracy of the news and the volume of the news sent. Therefore, the project had the following requirements:

- Reduce the time between the first detection of a news article until the story is deemed to meet the criteria for the relevant push notification.
- Remove, or at least reduce, the possibility of the same news story being pushed out more than once. This includes the same news article that are published by alternate sources.
- Ensure that the breaking news that is sent, better represents this type of news in contrast to the sending of more regular news items.
- Ensure that the news sent for user selected keywords, is of sufficient importance to be selected for the push notification. In other words, to avoid sending news articles that merely have a reference to the keyword. Instead the focus should be on sending news items that are centred around or have significant content related to the selected keyword.

Initially the use of machine learning was proposed to meet the requirements of the project. However, this was rejected by the client. Instead, with agreement of the client, an algorithm was developed to check for key indicators of breaking news and important news matching selected keywords. A method was also devised for the reading and processing of the articles supplied in the sample dataset. The team met fortnightly with Lexxe, to discuss the progress of the project and to make adjustments as required by the client. These adjustments ranged from choosing the best methods to classify the articles, to what should be displayed in the output for articles that met the set criteria. An initial prototype was also demonstrated to Lexxe where further requests were made by the client.
To validate the accuracy of our system, the Data Analysts from Group 19 and 20 were tasked with manually reading and classifying the news articles in the dataset. A final list was

then compiled containing all news articles that could be considered breaking news. For user selected keywords, each Data Analyst was also asked to select 5 keywords based on their prevalence in the articles they read. These requirements were also used as an aide for ongoing testing and analysis of the system where results were feedback to the Engineers to rectifying any problems detected.

A final report was supplied to Lexxe which detailed the articles output by our system and the accuracy for the breaking news and our 5 selected keywords.

Design

The design of this project has been a gradual process, as we have contended both with changes to the requirements we had to fulfill, as well as differences in opinion as to how we should fulfill said requirements.

It is using the value each team member provided however that we were able to draw upon each others strengths, and design a product that fulfilled the needs of the client. Many features saw continuous iterations as we both saw to improve our results, and keep up with shifting requirements, and in the end having delivered on both of these goals, it is clear that the reason we were successful was because of each other.

We split these incremental pushes into sprints. We planned on delivering versions of the product at key intervals, and in each interval we split the many different tasks we needed to complete into sprints. This made it easier to define not what work needed to be done, but also made it easier to see who was going to do what work, and when we could expect it to be delivered by. In all, this methodology made collaboration far smoother.

Implementation

Managing the implementation of even small software projects is no trivial task, with various competing theories on the best way organize developers to deliver a software solution. There are a lot of bad ways to do it, the most common one is to have a shared master branch, and a personal branch for each developer. The biggest problem with this sort of structure is the lack of organization; anybody can commit to the master branch. This can lead to “merge hell” and will kill productivity.

The most commonly featured workflow is GitFlow[1] and a variant Github Flow[2], which is the workflow used by github to develop and deploy their software. Unfortunately, both of these workflows are too extensive to describe in detail here, but i shall summarize the main point briefly. For more information see the link in the footnotes on this page.

[1] https://datasift.github.io/gitflow/IntroducingGitFlow.html
[2] https://guides.github.com/introduction/flow/

In short GitFlow is a framework for organizing git source control repositories. It creates a highly structured set of branch conventions, rules and conditions that are enforced. There is a main “develop” branch this functions as the target for current work. However, no developer is allowed to commit to this branch, instead they will create “feature” branches. When a developer starts work on a new feature they will create a new features branch from the develop branch. All work on this features happens in this branch. Once work is complete, they will create a “pull request” (a code review) to merge the changes into the develop branch. This ensures that all code that deployed to production is always tested.

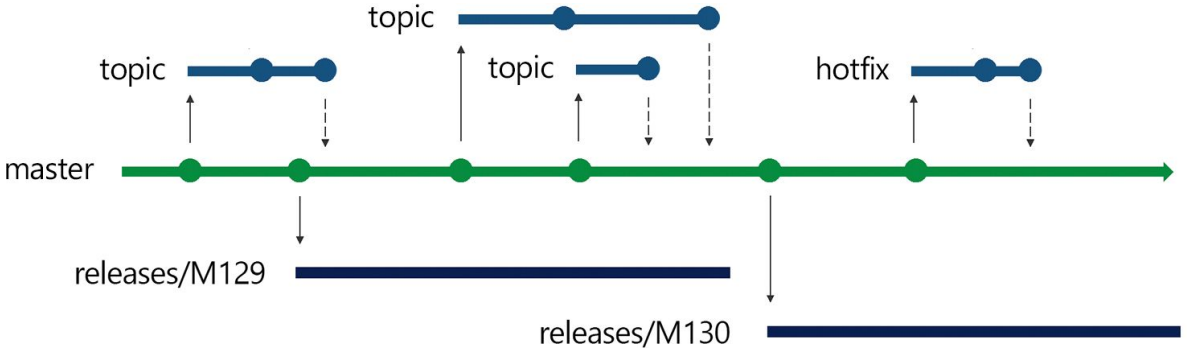

Figure 1: Release Flow (Source Edward Thomson[3]).

In this project we used a variant of Github Flow called Release Flow. From Figure 1 we can see that Release Flow is still a trunk based workflow but feature branches are called topic branches and the main branch is master instead of develop. The differences comes the way that releases are handled, we can see in Figure 1 that there are two releases M129 and M130 and each of these came from a separate sprint.
Github flow focuses heavily on continuous deployment, in which every single commit in the main branch is deployed to production. This is unfortunately not practical in this project because there is a cost to the client for each release that it integrated. Release Flow takes a more practical approach to continuous deployment, with a new release being created after each sprint. This approach had a more natural fit, we had 3 sprint corresponding to each increment and a release for each one as well.

On the whole this approach worked very well, and we had zero difficult merges conflicts to resolve which we attribute directly to this methodology. We did hit a few bumps along the road, the most significant was the requirement that all code must be reviewed before it is merged into the main branch coupled with the fact that a new feature branches off the current develop branch. It might not be obvious why this could create a problem but given the schedules of different developers, it could take a few days for the branch you have been working on to get merged into the develop branch.
This creates a problem if feature you are working on now requires the one you just finished. To follow follow this scheme to the letter, it would mean that you could not work on the new features until someone had enough time to review those changes. In the times this issue

3 https://blogs.msdn.microsoft.com/devops/2018/04/19/release-flow-how-we-do-branching-on-the-vsts-team/

came up we bent the rules, instead of branching of the most recent develop branch we branch of the feature branch we just finished. Once first branch had been merged into the main branch we would rebase those changes to follow the guidelines. It's wasn't perfect and i would not want to do it on larger projects but it did not cause any issues.

As i have mentioned one to the key focuses for both Github Flow and Release Flow is both continuous integration (CI). Continuous Integration means different things depending on the project, but the most fundamental part is that all of the parts that make up the project are brought together (integrated) and tested to make sure the whole product works. This is done for every change to the code (continuously) regardless of how small. This can be an absolutely daunting task, but thankfully there are a number of services that make this much more manageable. The provider that we chose was CircleCI, which has very nice integration with Github (where we hosted the source code) and provides a free tier that was adequate for our needs.

When everything goes right, you don't need to lift a finger (once it is set up that is) because everything is automated. In Figure 2 we can see the three most recent commits have successfully passed all unit tests and build correctly.

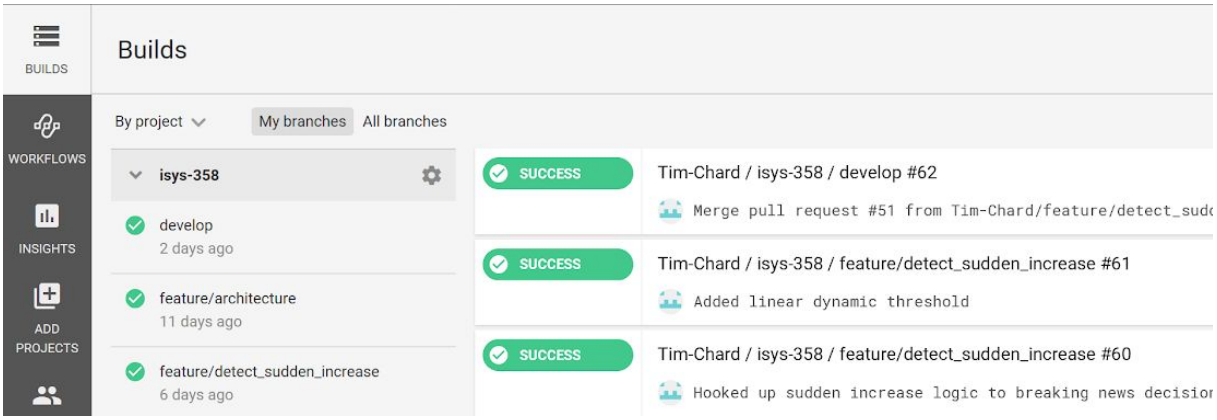

Figure 2: CircleCI continuous integration environment showing a healthy build history.

The real magic of continuous integration is when things don't go right. As soon as a build fails, the author of the commit gets an email (such as the one in Figure 3) notifying them of the problem.

Figure 3: CircleCI early warning notification.

The importance of these email can *not* be understated because the time taken to fix a bug is directly related to the time that the bug has been in the software. It is not uncommon to hear stories of developers spending days to track down a bug that is fixed by removing or adding a few characters. A misplaced semicolon that has been in the software for months will be incredible hard to track down, but would have take mere minutes if the person who introduced it was aware of the problem while the code was still fresh in his mind.

These early warning notifications saved us from similar situation on more than one occasion, and i am firmly convinced that every single non-trivial project should spend the time to implement a continuous integration workflow.

Learning Outcomes

Teamwork
This PACE unit taught us how to work cohesively as a team. One of the ways this was achieved was through task allocation. Our members consisted of Project Managers, Data Analysts, and Programmers. By assigning the tasks that best matches each member's skill set we created a better product overall.

Project Management
Project management was an integral part of the system's development. It helped to allocate tasks to the appropriately skilled individuals, as well as manage our time effectively. We learned how to balance a team oriented project alongside other personal obligations.

Client/Sponsor
This unit allowed us to interact with a real client and develop an agile solution based on their requirements. After fortnightly consultations with client, we found that some additional functionality was requested. Finding this out before the project was completed is why the

agile approach is so versatile. Our group was up to the challenge, and we updated the prototype to match the new requirements he requested. Towards the end of development, the client asked for some additional functionality. Due to the unit's time constraints, it was decided that the extra functionality would be handled by the successive year's PACE groups.

Test data
The creation of test data was a joint project between our group and Group 20. This required inter-team communication to create clean and consistent data. The test data in this case is identifying which articles are breaking news. Our data analyst played a major role in keeping the data consistent, by thoroughly defining the definition of breaking news.

Skills learned from past units
There were some units that one or multiple team members found directly helpful for this project. Two examples are: *COMP348* for Natural Language Processing of articles, and *COMP336* to implement MongoDB to store and retrieve articles. These and other foundation units gave the team a large skill set to tackle the challenges faced in this project.

Conclusion

This PACE unit has provided an unique opportunity to gain practical experience in software development. It grouped together a team of students with different skill sets to collaboratively solve a problem for a client. With the team's usage of the agile development methodology, our provided system was able to match up to the clients requirements as they changed.

It was a learning experience to apply the agile approach in a non-theoretical scenario. This involved the use of agile sprint management and rigorous code testing. The sprint management came in the form of Release Flow, a program that manages version control, as well as managing the prototypes created from each sprint.
To ensure that each sprint resulted in error-free, functional code, we used CircleCI. This is a form of continuous integration, which ensures that all functional code is integrated into the prototype, and that every change to the code is continuously tested for errors.

As of this report, we have had 3 sprints corresponding to each increment and a release (prototype) for each one as well.

The result of our efforts is a feature complete system that meets all the functionality requirements specified by the sponsor. This includes exceeding the required accuracy of 60% for our Breaking News Classifier. We have received positive feedback during a series of meetings to demonstrate the prototype.